

Assignment 7

Aim

Implement hashing, use a menu driven options of linear probing, quadratic probing and double hashing for collision resolution.
Count the average number of times a location is calculated for entering a key using the given techniques.

Code

```
#include <stdio.h>
#define MAX 100
```

```
int hashTable [MAX];
int tableSize;
```

```
int hashFunction (int key) {
    return key % tableSize;
}
```

```
int secondHashFunction (int key) {
    return 1 + (key % (tableSize - 1));
}
```

```
void LinearProbing (int key) {
    int index;
    int h = hashFunction (key);
    int locations = 0;
    for (int i = 0; i < tableSize; i++) {
        locations++;
        index = (h + i) % tableSize;
        if (hashTable [index] == -1) {
            hashTable [index] = key;
        }
    }
}
```

```

    printf("Key %d inserted at index %d\n",
           key, index);
    printf("Locations calculated: %d\n", locations);
    return;
  }
}

printf("Hash table is full!\n");
}

void quadraticProbing(int key)
{
  int index;
  int h = hashFunction(key);
  int locations = 0;
  for (int i = 0; i < tableSize; i++) {
    locations++;
    index = (h + i) % tableSize;
    if (hashTable[index] == -1) {
      hashTable[index] = key;
      printf("Key %d inserted at index %d\n", key, index);
      printf("Locations calculated: %d\n", locations);
      return;
    }
  }
  printf("Hash table is full!\n");
}

```

```

void doubleHashing(int key){
    int index;
    int h1 = hashFunction(key);
    int h2 = hashFunction(key);
    int locations = 0;
    for (int i=0; i < tableSize; i++){
        locations++;
        index = (h1 + i * h2) % tableSize;
        if (hashTable[index] == -1){
            hashTable[index] = key;
            printf("key %d inserted at index\n", key, index);
            printf("Locations calculated: %d\n", locations);
            return;
        }
    }
    printf("Hash table is full!\n");
}
  
```

```

void insertKey(int keys[], int n, int choice){
    int totalLocations = 0;
    int before, after;
    for (int i=0; i < n; i++){
        before = 0;
        if (choice == 1){
            int key = keys[i];
            int h = hashFunction(key);
            for (int j=0; j < tableSize; j++){
  
```



```

int index = (h + j) % tableSize;
if (hashTable[index] == -1) {
    before = j + 1;
    break;
}
}
}
linearProbing(key);
}
elseif (choice == 2) {
    int key = keys[i];
    int h = hashFunction(key);
    for (int j = 0; j < tableSize; j++) {
        int index = (h + j * j) % tableSize;
        if (hashTable[index] == -1) {
            before = j + 1;
            break;
        }
    }
}
quadraticProbing(key);
}
else if (choice == 3) {
    int key = key[i];
    int h1 = hashFunction(key);
    int h2 = secondHashFunction(key);
    for (int j = 0; j < tableSize; j++) {
        int index = (h1 + j * h2) % tableSize;
        if (hashTable[index] == -1) {
            before = j + 1;
            break;
        }
    }
}
}

```

```

    double Hashing (key);
}
totalLocations += before;
}
printf("\n Hash Table: \n");
for (int i=0; i<tableSize; i++) {
    if (hashTable[i] == -1) {
        printf("Index %d: EMPTY \n", i);
    }
    else {
        printf("Index %d : %d \n", i, hashTable[i]);
    }
}
printf("\n Total Locations calculated = %d \n",
totalLocations);
printf(" Number of keys inserted = %d \n", n);
printf(" Average locations calculated = %.2f \n",
(float) totalLocations / n);
}

void main () {
    int keys[MAX];
    int n;
    int choice;
    printf("Enter hash table size: ");
    scanf("%d", &tableSize);
    if (tableSize <= 1 || tableSize > MAX) {
        printf("Invalid table size! \n");
        return;
    }
    printf("Enter numbers of keys: ");
    scanf("%d", &n);

```

```

if (n > tableSize || n <= 0) {
    printf("Invalid number of keys!\n");
    return;
}

printf("Enter %d keys:\n", n);
for (int i = 0; i < n; i++) {
    scanf("%d", &key[i]);
}

do {
    printf("1 - Linear Probing\n");
    printf("2 - Quadratic Probing\n");
    printf("3 - Double Hashing\n");
    printf("4 - Exit\n");
    printf("Enter your choice: ");
    scanf("%d", &choice);
    if (choice >= 1 && choice <= 3) {
        for (int i = 0; i < tableSize; i++) {
            hashTable[i] = -1;
        }
        printf("\n");
        insertKeys(keys, n, choice);
    }
    else if (choice == 4) {
        printf("Program Terminated.\n");
    }
    else {
        printf("Invalid choice!\n");
    }
} while (choice != 4);

```


Enter hash table size: 10
Enter number of keys: 7

Enter 7 keys:

23
43
13
27
33
53
63

1 - Linear Probing
2 - Quadratic Probing
3 - Double Hashing
4 - Exit

Enter your choice: 1

Key 23 inserted at index 3
Locations calculated: 1
Key 43 inserted at index 4
Locations calculated: 2
Key 13 inserted at index 5
Locations calculated: 3
Key 27 inserted at index 7
Locations calculated: 1
Key 33 inserted at index 6
Locations calculated: 4
Key 53 inserted at index 8
Locations calculated: 6
Key 63 inserted at index 9
Locations calculated: 7

Hash Table:

Index 0 : EMPTY
Index 1 : EMPTY
Index 2 : EMPTY
Index 3 : 23
Index 4 : 43
Index 5 : 13
Index 6 : 33
Index 7 : 27
Index 8 : 53
Index 9 : 63

Total locations calculated = 24

Total locations calculated = 24
Number of keys inserted = 7
Average locations calculated = 3.43
1 - Linear Probing
2 - Quadratic Probing
3 - Double Hashing
4 - Exit

Enter your choice: 2

Key 23 inserted at index 3
Locations calculated: 1
Key 43 inserted at index 4
Locations calculated: 2
Key 13 inserted at index 7
Locations calculated: 3
Key 27 inserted at index 8
Locations calculated: 2
Key 33 inserted at index 2
Locations calculated: 4
Key 53 inserted at index 9
Locations calculated: 5
Hash table is full!

Hash Table:

Index 0 : EMPTY
Index 1 : EMPTY
Index 2 : 33
Index 3 : 23
Index 4 : 43
Index 5 : EMPTY
Index 6 : EMPTY
Index 7 : 13
Index 8 : 27
Index 9 : 53

Total locations calculated = 17
Number of keys inserted = 7
Average locations calculated = 2.43
1 - Linear Probing
2 - Quadratic Probing
3 - Double Hashing
4 - Exit
Enter your choice: 3

Key 23 inserted at index 3
Locations calculated: 1

Number of keys inserted = 7
Average locations calculated = 2.43
1 - Linear Probing
2 - Quadratic Probing
3 - Double Hashing
4 - Exit
Enter your choice: 3

Key 23 inserted at index 3
Locations calculated: 1
Key 43 inserted at index 1
Locations calculated: 2
Key 13 inserted at index 8
Locations calculated: 2
Key 27 inserted at index 7
Locations calculated: 1
Key 33 inserted at index 0
Locations calculated: 2
Key 53 inserted at index 2
Locations calculated: 2
Key 63 inserted at index 4
Locations calculated: 2

Hash Table:

Index 0 : 33
Index 1 : 43
Index 2 : 53
Index 3 : 23
Index 4 : 63
Index 5 : EMPTY
Index 6 : EMPTY
Index 7 : 27
Index 8 : 13
Index 9 : EMPTY

Total locations calculated = 12
Number of keys inserted = 7
Average locations calculated = 1.71
1 - Linear Probing
2 - Quadratic Probing
3 - Double Hashing
4 - Exit
Enter your choice: 4

Program terminated.

✧ tarun@Tarun-PC:~/Downloads/ADSA-TSEC-SEM3\$